Lista 13

Marta Hałas

2025-01-26

Spis treści

1 sekcja 1 **1**
1.1 zadanie 1 1
1.2 zadanie 2 3

2 sekcja 2 **4**
2.1 zadanie 1 4
2.2 zadanie 2 6

3 sekcja 3 **7**
3.1 zadanie 1 7
3.2 zadanie 2 10

1 sekcja 1

1.1 zadanie 1

a)

```
f<-expression(sin(10*x) -exp(-x))
D(f,"x")
```

```
## cos(10 * x) * 10 + exp(-x)
```

Pochodna jest prawidłowa.

b)

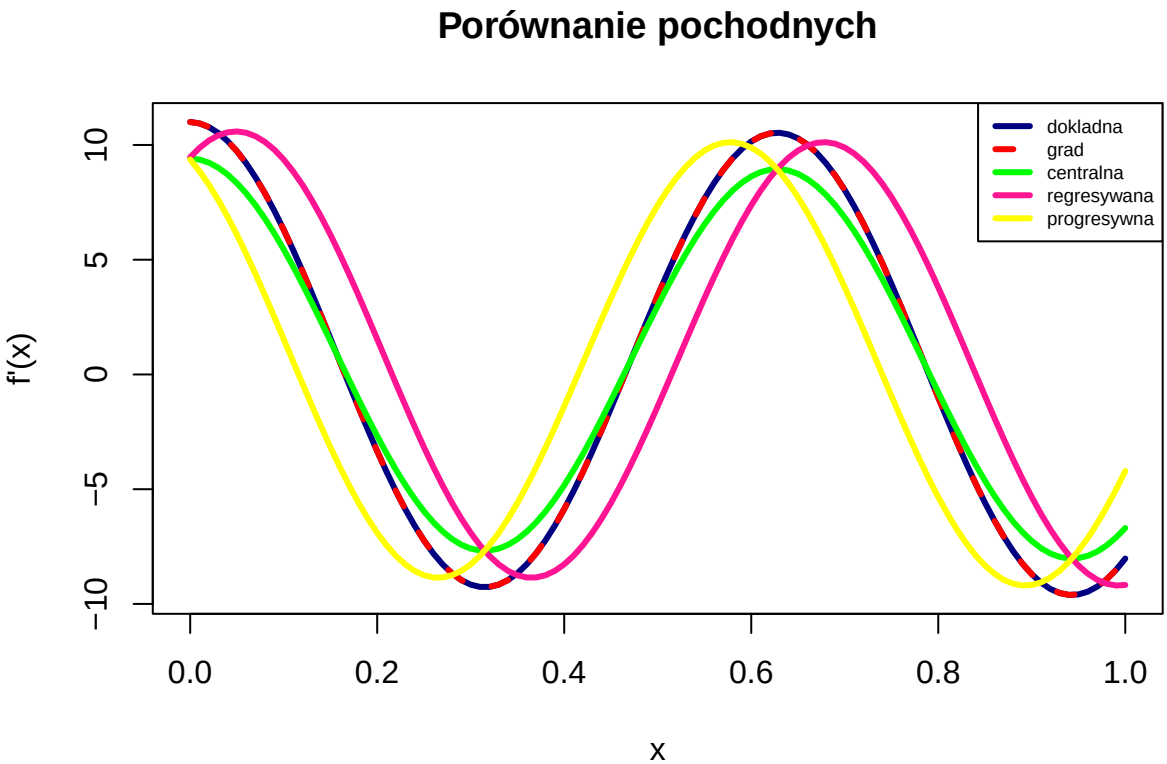
```
#dokładna
f<-expression(sin(10*x) -exp(-x))
g<-D(f,"x")
gfun <- function(x) eval(g)
```

```r
h=0.1
x <- seq(0, 1, 0.01)

library(numDeriv)
f <- function(x) sin(10*x) -exp(-x)
df.numer <- function(x) grad(f, x)
df.numeric<- sapply(x, df.numer)
#progresywana
p <- function(x) {
  (f(x + h) - f(x)) / h
}
p <- sapply(x, p)
#regresywna
r <- function(x) {
  (f(x) - f(x - h)) / h
}
r <- sapply(x, r)
#centralna
cr<- function(x) {
  (f(x + h) - f(x - h)) / (2 * h)
}
cr <- sapply(x, cr)



plot(x, gfun(x), type = "l", col = "navy", lwd = 3,
     xlab = "x", ylab = "f'(x)", main = "Porównanie pochodnych")
lines(x, df.numeric, col = "red", lwd = 3, lty=2)
lines(x, cr, col = "green", lwd = 3)
lines(x, r, col = "deeppink", lwd = 3)
lines(x, p, col = "yellow", lwd = 3)
legend("topright", legend=c("dokładna","grad","centralna","regresywana", "progresywna"),
       col=c("navy", "red", "green","deeppink", "yellow"), cex=0.6, lwd=c(3,3,3,3,3),
lty=c(1,2,1,1,1))
```



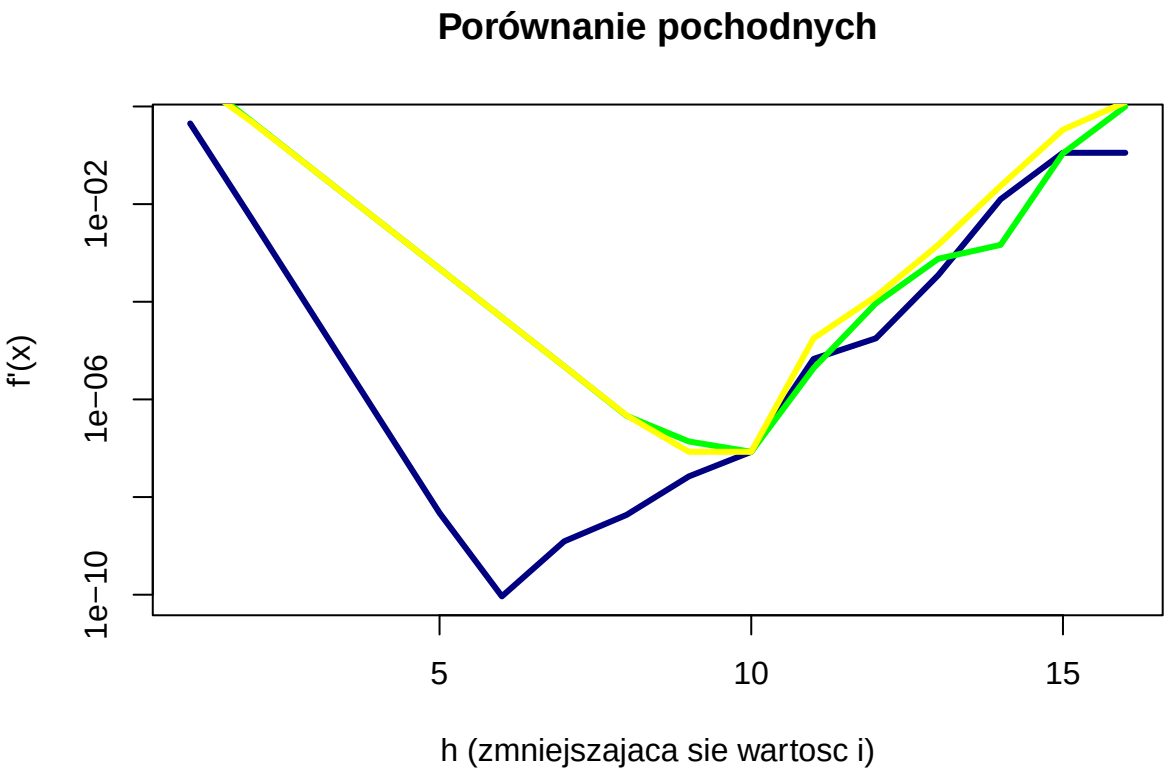
1.2 zadanie 2

```
f<-function(x) sin(10*x) -exp(-x)
g <- D(expression(sin(10 * x) - exp(-x)), "x")
gfun <- function(x) eval(g)

i<-seq(1,16,1)
h=10^(-i)
x0=0.5
#progresywana
p <- function(h,x) { abs(
  (f(x + h) - f(x)) / h -gfun(x))
}
p <- sapply(h, FUN= p, x=x0)
#regresywna
r <- function(h,x) {abs(
  (f(x) - f(x - h)) / h -gfun(x))
}
r <- sapply(h, FUN=r,x=x0)
#centralna
cr<- function(h,x) { abs(
  (f(x + h) - f(x - h)) / (2 * h) -gfun(x))
}
cr<- sapply(h, FUN= cr, x=x0)

plot(i, cr, type = "l", col = "navy", lwd = 3,log="y",
     xlab = "h (zmniejszająca się wartość i)", ylab = "f'(x)", main = "Porównanie pochodnych")
```

```
lines(i, r, col = "green", lwd = 3)
lines(i, p, col = "yellow", lwd = 3)
```



Najbardziej optymalna jest wartość kroku h dla i =6 oraz i= 10.

2 sekcja 2

2.1 zadanie 1

a)

```
#i
library(cmna)
f1<- function(x) x-x^2
dokładna<- 1/6
dokładna
```

```
## [1] 0.1666667
```

```
n<-c(10,100,1000)
szacowane <- function(f, a, b, n) {
  prostokąt <- midpt(f, a, b, m = n)
  trapez <- trap(f, a, b, m = n)
  simp <- simp(f, a, b, m = n)

  return(c(prostokąt, trapez, simp))
```

```r
}

wyniki <- sapply(n, function(n) szacowane(f, 0, 1, n))
as.data.frame(wyniki)
```

```
##           V1         V2         V3
## 1 -0.4400578 -0.4481341 -0.4482126
## 2 -0.4643273 -0.4483720 -0.4482150
## 3 -0.4481476 -0.4482134 -0.4482134
```

```r
colnames(wyniki) <- n
rownames(wyniki) <- c("prostokąt", "trapez", "simp")
wyniki
```

```
##                   10        100       1000
## prostokąt -0.4400578 -0.4481341 -0.4482126
## trapez    -0.4643273 -0.4483720 -0.4482150
## simp      -0.4481476 -0.4482134 -0.4482134
```

```r
#ii
f2<- function(x) exp(x)*cos(x)
dokładna<- -(exp(pi)+1)/2
dokładna
```

```
## [1] -12.07035
```

```r
n<-c(10,100,1000)
szacowane <- function(f, a, b, n) {
  prostokąt <- midpt(f, a, b, m = n)
  trapez <- trap(f, a, b, m = n)
  simp <- simp(f, a, b, m = n)

  return(c(prostokąt, trapez, simp))
}

wyniki <- sapply(n, function(n) szacowane(f, 0, pi, n))
as.data.frame(wyniki)
```

```
##           V1         V2         V3
## 1 -0.9528628 -0.9567467 -0.9567857
## 2 -0.9646424 -0.9568648 -0.9567869
## 3 -0.9567893 -0.9567861 -0.9567861
```

```r
colnames(wyniki) <- n
rownames(wyniki) <- c("prostokąt", "trapez", "simp")
wyniki
```

```
##                   10        100       1000
## prostokąt -0.9528628 -0.9567467 -0.9567857
## trapez    -0.9646424 -0.9568648 -0.9567869
## simp      -0.9567893 -0.9567861 -0.9567861
```

Najszybciej działa algorytm simp.

b)

```
library(cmna)
dokładna<- -(exp(pi)+1)/2
dokładna1<- 1/6
dokładna1 - integrate(f1,0,1)$value
```

```
## [1] 0
```

```
dokładna - integrate(f2,0,pi)$value
```

```
## [1] 1.776357e-15
```

Błędy są niewielkie w przypadku f2, a w przypadku f1 wynoszą 0.

2.2 zadanie 2

```
k<-seq(1,20,1)
  g <- function(x) {
  ifelse(x >= 0 & x <= 1, 1, 0)
}

całka <- function(a,b) {
  i<-integrate(g,a,b)$value
  return (i)
}

wynik<-sapply(k, function(k) całka(-k,10))
wynik
```

```
##  [1] 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000
##  [8] 1.0000000 0.9999962 1.0000000 0.0000000 1.0000000 1.0000090 0.0000000
## [15] 0.0000000 1.0000000 1.0000260 1.0000000 0.0000000 0.0000000
```

```
integrate(g,-2,4, subdivisions=100)$value
```

```
## [1] 1
```

```
integrate(g,-2,4, rel.tol=0.0000000000001,subdivisions=100 )$value
```

```
## [1] 1
```

```
integrate(g, -2,4, abs.tol=1)$value
```

```
## [1] 1.023928
```

```r
integrate(g,-6,4, subdivisions=100)$value
```

```
## [1] 1
```

```r
integrate(g,-6,4, rel.tol=0.0000000000001,subdivisions=100 )$value
```

```
## [1] 1
```

```r
integrate(g, -6,4, abs.tol=1)$value
```

```
## [1] 0.9993893
```

Nie, wyniki wahają się w okolicach 1, jednak zauważamy też kilka zer. Zmiana parametru na abs.tol daje nam dokładniejszy wynik.

3 sekcja 3

3.1 zadanie 1

a)

```r
library(polynom)
f <- function (x) {
  exp(-3*x)*cos(4*pi*x)
}

n<-7
x<-seq(0,1,1/n)
y<-sapply(x,FUN=f)

f.p<-as.function(poly.calc(x,y))
is.function(f.p)
```

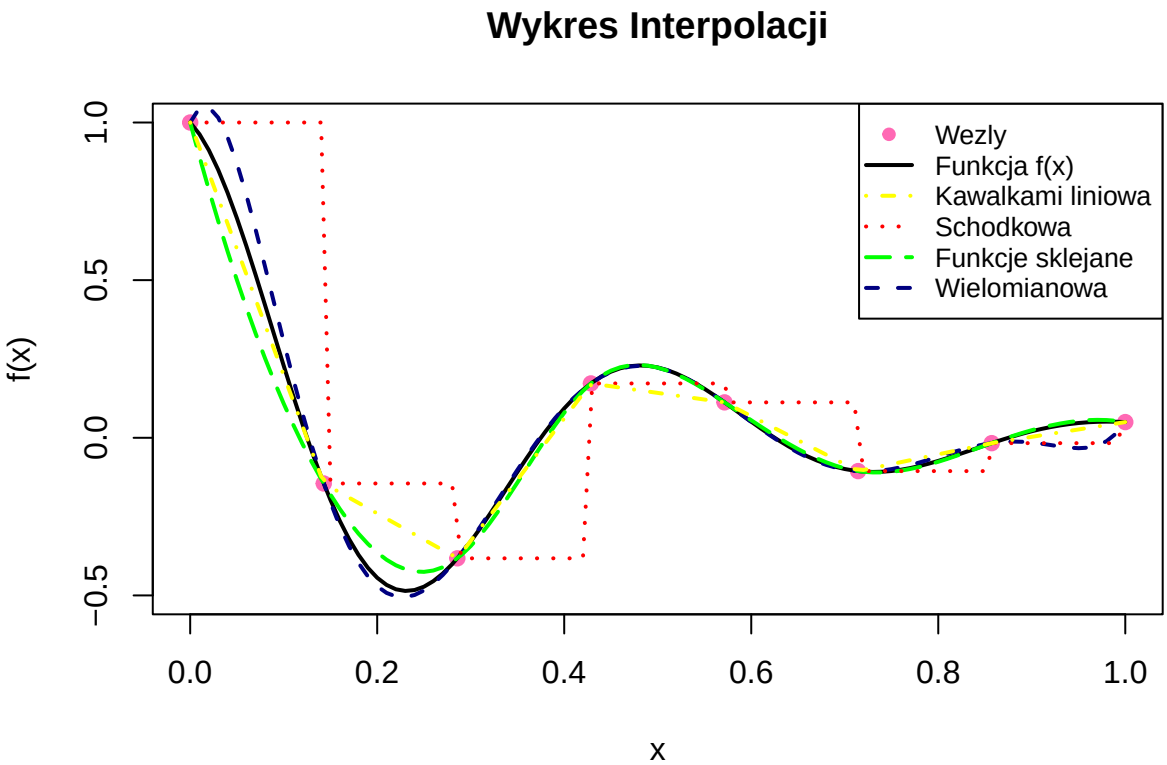
```
## [1] TRUE
```

```r
f.s<-splinefun(x,y)

f.const<-approxfun(x,y, method="constant")
f.linear<-approxfun(x,y, method="linear")


plot(x,y, col="hotpink", pch=19, main="Wykres Interpolacji", ylim=c(-0.5,1), ylab="f(x)")
curve(f, add=TRUE, lwd=2)
curve(f.p, add=TRUE, lwd=2, lty=2, col="navy")
curve(f.s, add=TRUE, lwd=2, lty=5, col="green")
curve(f.const, add=TRUE, col="red", lty=3, lwd=2)
curve(f.linear, add=TRUE, col="yellow", lty=4, lwd=2)
legend("topright",
```

```
legend=c("Węzły","Funkcja f(x)","Kawałkami liniowa", "Schodkowa", "Funkcje sklejane",
         "Wielomianowa"),
       col=c("hotpink", "black", "yellow", "red","green", "navy"),
cex=0.8,
lwd=c(NA,2,2,2,2,2),
lty=c(NA,1,4,3,5,2), pch=c(19,NA,NA,NA,NA,NA))
```



b)

```
library(pracma)
f <- function (x) {
  exp(-3*x)*cos(4*pi*x)
}
n<- 5
x<- seq(0,1,1/n)
y<-sapply(x,FUN=f)

f1<-approxfun(x,y, method="constant")
f2<- approxfun(x,y, method="linear")
f3<- as.function(poly.calc(x,y))
f4<-splinefun(x,y)
odl <- function(g){
  x_max<- findmins(function(x) (-abs(f(x)-g(x))),0,1)
  glob <- max(sapply(x_max,FUN = function(x) (abs(f(x)-g(x)))))
  return(glob)
}
odl(f1)
```

```
## [1] 1.443835
```

```r
odl(f2)
```

```
## [1] 0.1748594
```

```r
odl(f3)
```

```
## [1] 0.6318571
```

```r
odl(f4)
```

```
## [1] 0.4275442
```

```r
n<- 10
x<- seq(0,1,1/n)
y<-sapply(x,FUN=f)

f1<-approxfun(x,y, method="constant")
f2<- approxfun(x,y, method="linear")
f3<- as.function(poly.calc(x,y))
f4<-splinefun(x,y)
odl <- function(g){
  x_max<- findmins(function(x) (-abs(f(x)-g(x))),0,1)
  glob <- max(sapply(x_max,FUN = function(x) (abs(f(x)-g(x)))))
  return(glob)
}
odl(f1)
```

```
## [1] 0.7705039
```

```r
odl(f2)
```

```
## [1] 0.09009446
```

```r
odl(f3)
```

```
## [1] 0.02327858
```

```r
odl(f4)
```

```
## [1] 0.04579354
```

```r
n<- 15
x<- seq(0,1,1/n)
y<-sapply(x,FUN=f)

f1<-approxfun(x,y, method="constant")
f2<- approxfun(x,y, method="linear")
f3<- as.function(poly.calc(x,y))
f4<-splinefun(x,y)
```

```r
odl <- function(g){
  x_max<- findmins(function(x) (-abs(f(x)-g(x))),0,1)
  glob <- max(sapply(x_max,FUN = function(x) (abs(f(x)-g(x)))))
  return(glob)
}
odl(f1)
```

```
## [1] 0.6177362
```

```r
odl(f2)
```

```
## [1] 0.05322484
```

```r
odl(f3)
```

```
## [1] 9.431215e-05
```

```r
odl(f4)
```

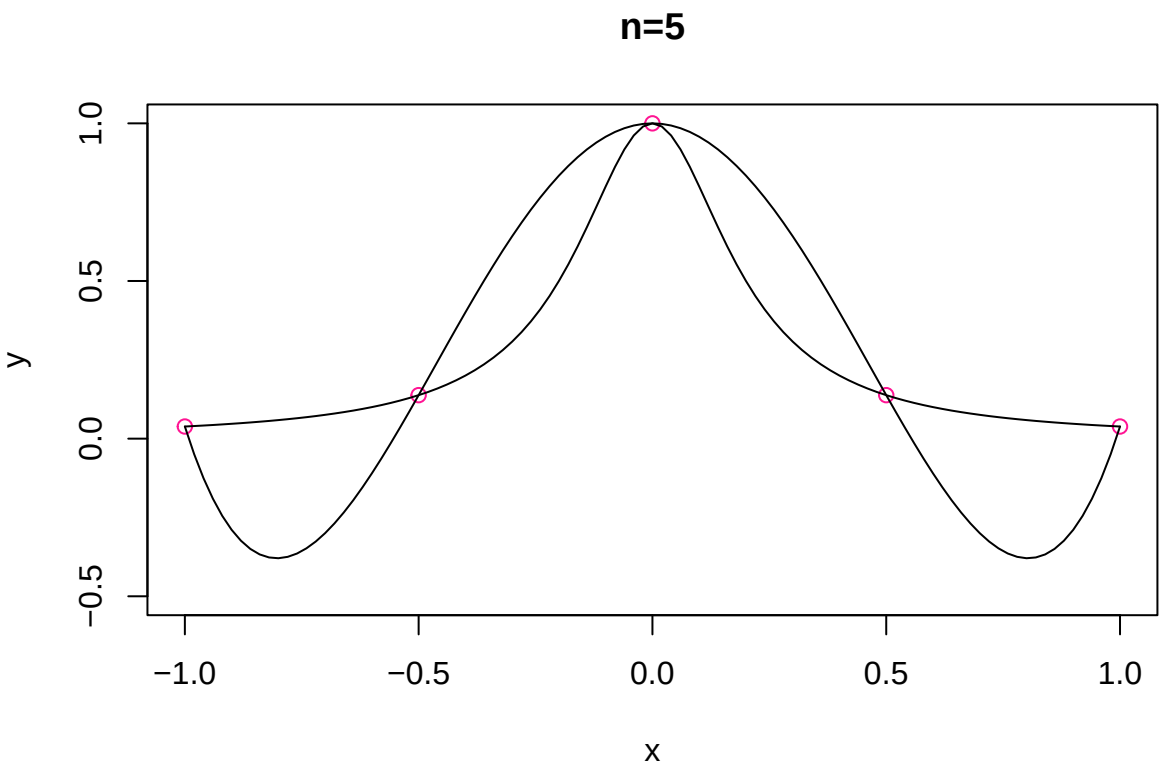
```
## [1] 0.005075408
```

W przypadku n=5 możemy uznać interpolacje za zadowalającą.

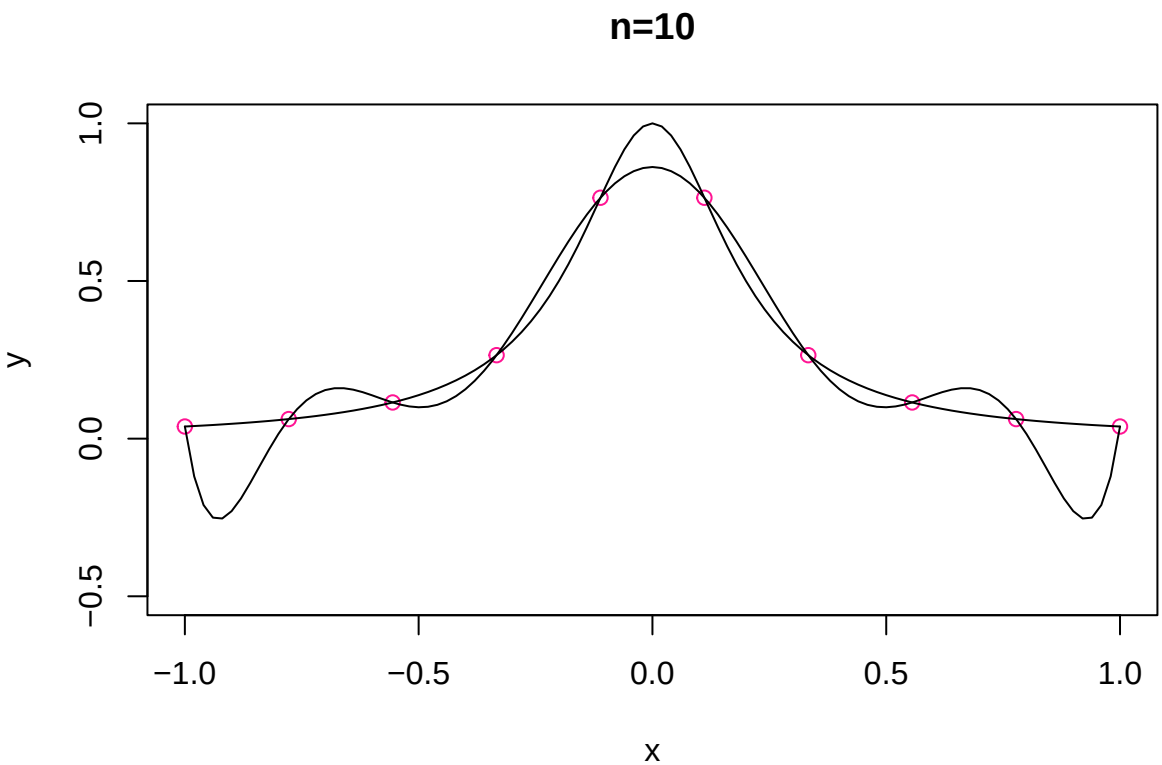
3.2 zadanie 2

a)

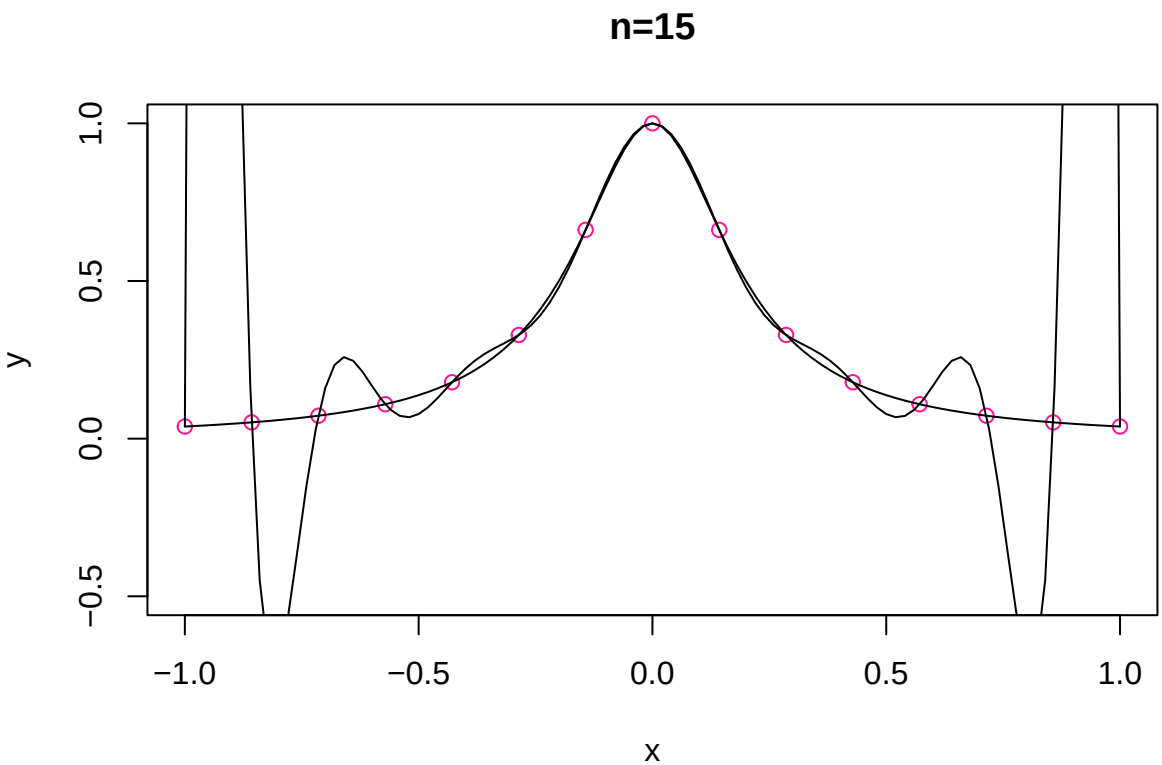
```r
f<- function(x) 1/(1+25*(x^2))
n<-5
x<-seq(-1,1,length.out=n)
y<-sapply(x,FUN=f)
f.p<-as.function(poly.calc(x,y))
plot(x,y,col="deeppink", main="n=5", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.p, add=TRUE)
```




```
n<-10
x<-seq(-1,1,length.out=n)
y<-sapply(x,FUN=f)
f.p<-as.function(poly.calc(x,y))
plot(x,y,col="deeppink", main="n=10", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.p, add=TRUE)
```



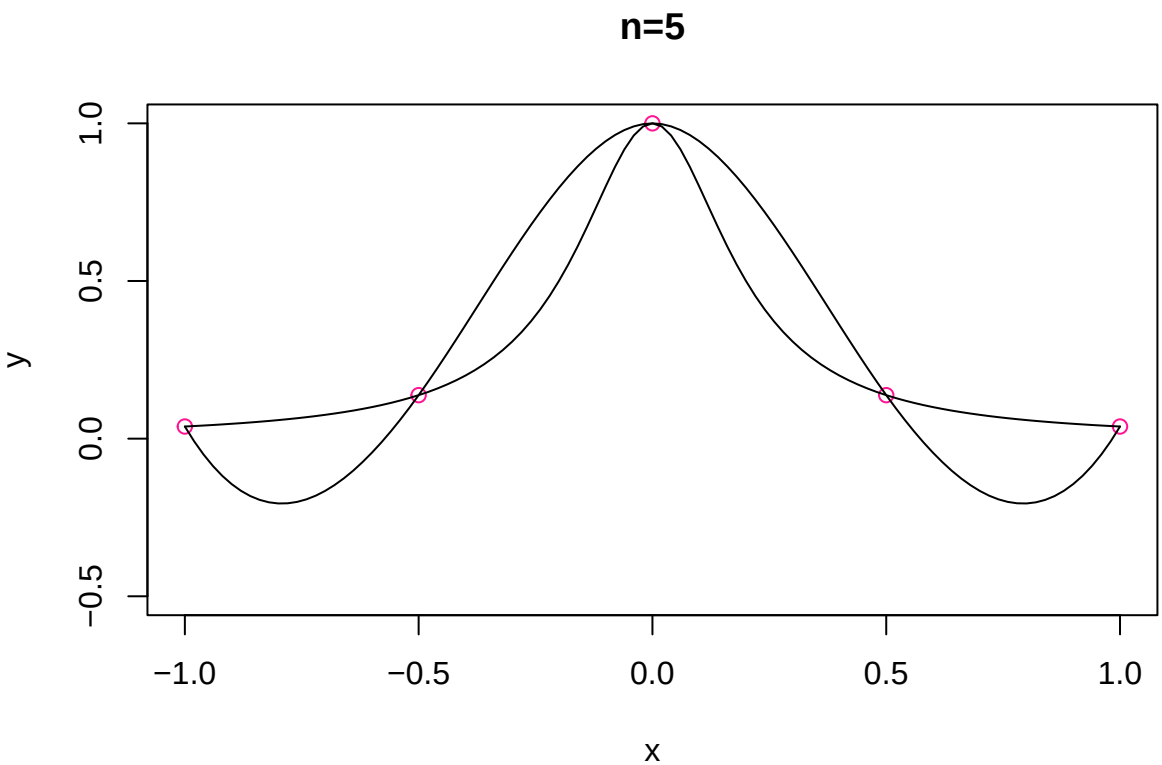

```
n<-15
x<-seq(-1,1,length.out=n)
y<-sapply(x,FUN=f)
f.p<-as.function(poly.calc(x,y))
plot(x,y,col="deeppink", main="n=15", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.p, add=TRUE)
```



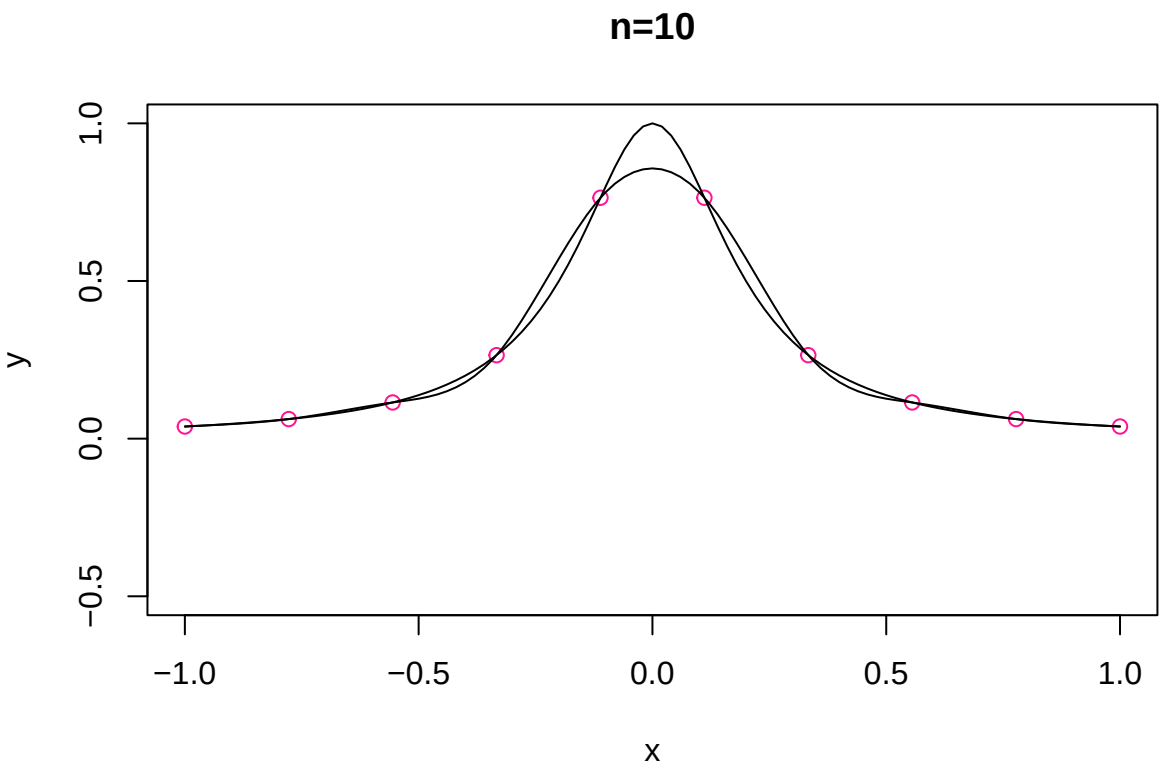
Najlepsze interpolacje funkcji są dla x od -0.5 do 0.5, Najgorsze na brzegach przedziału. Im większe n tym lepsze przybliżenia na odc, (-0.5,0.5), a gorsze na brzegach.

b)

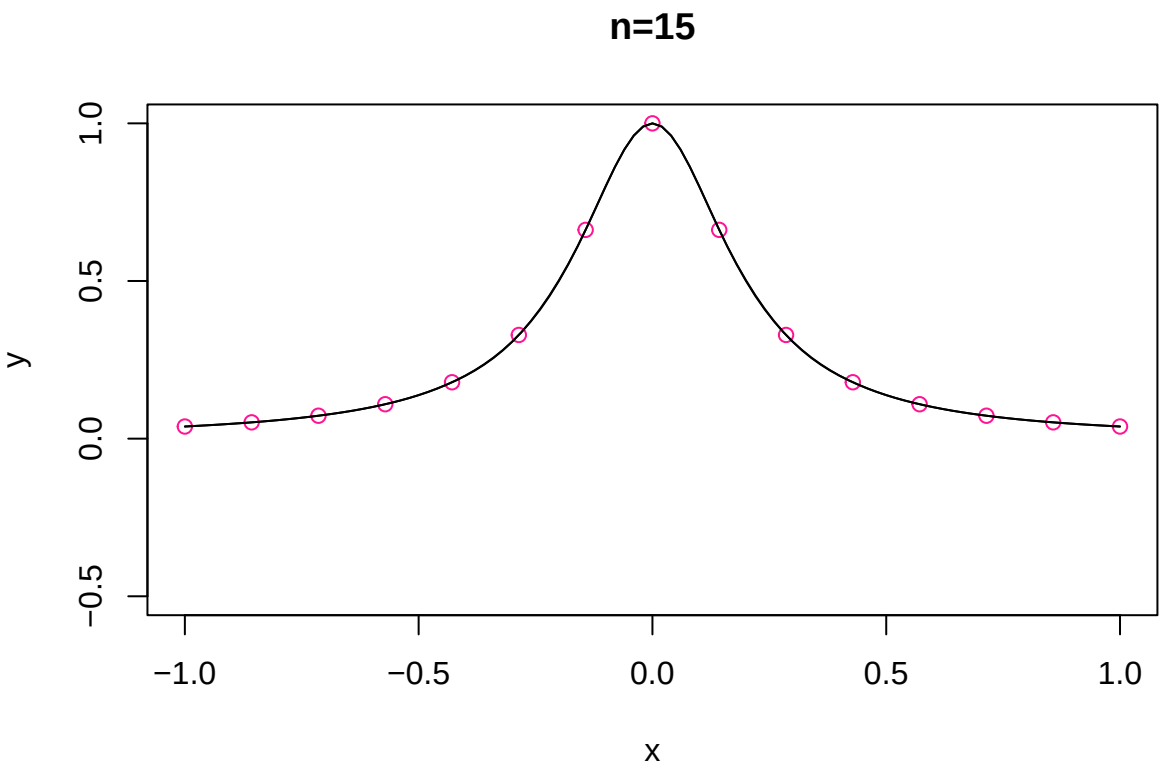
```
f<- function(x) 1/(1+25*(x^2))
n<-5
x<-seq(-1,1,length.out=n)
y<-sapply(x,FUN=f)
f.s<-splinefun(x,y)
plot(x,y,col="deeppink", main="n=5", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.s, add=TRUE)
```




```
n<-10
x<-seq(-1,1,length.out=n)
y<-sapply(x,FUN=f)
f.s<-splinefun(x,y)
plot(x,y,col="deeppink", main="n=10", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.s, add=TRUE)
```



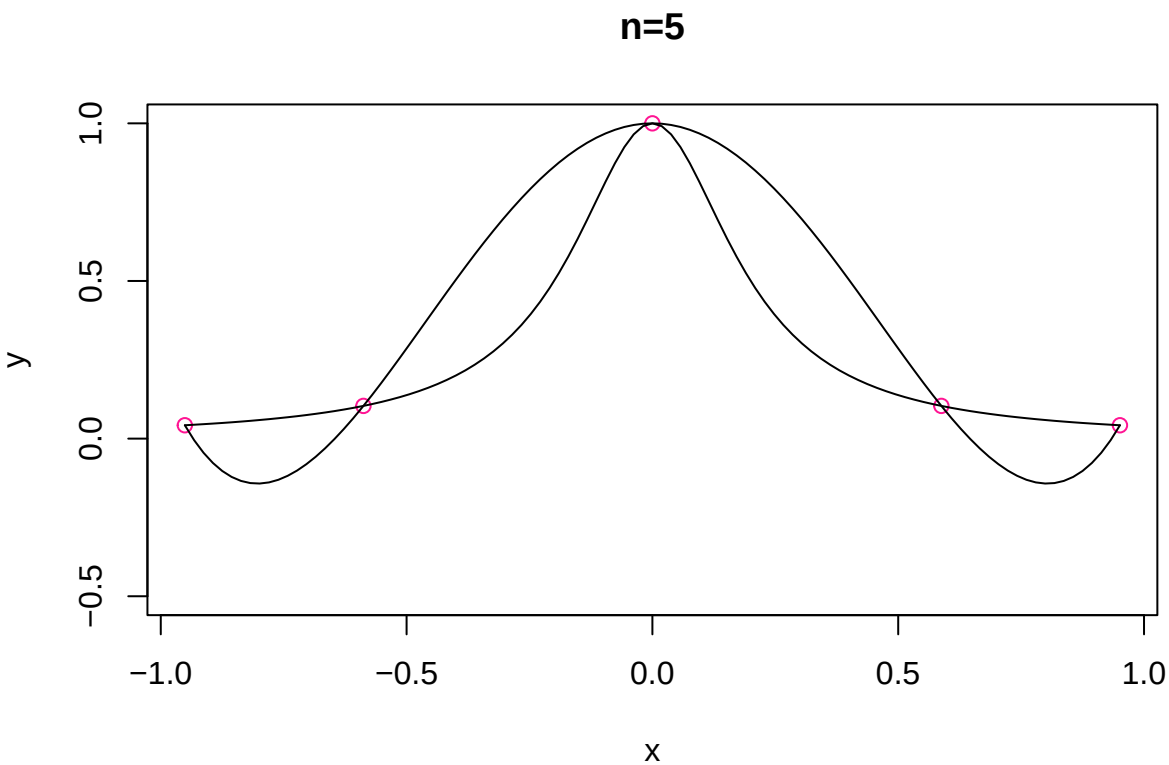

```
n<-15
x<-seq(-1,1,length.out=n)
y<-sapply(x,FUN=f)
f.s<-splinefun(x,y)
plot(x,y,col="deeppink", main="n=15", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.s, add=TRUE)
```



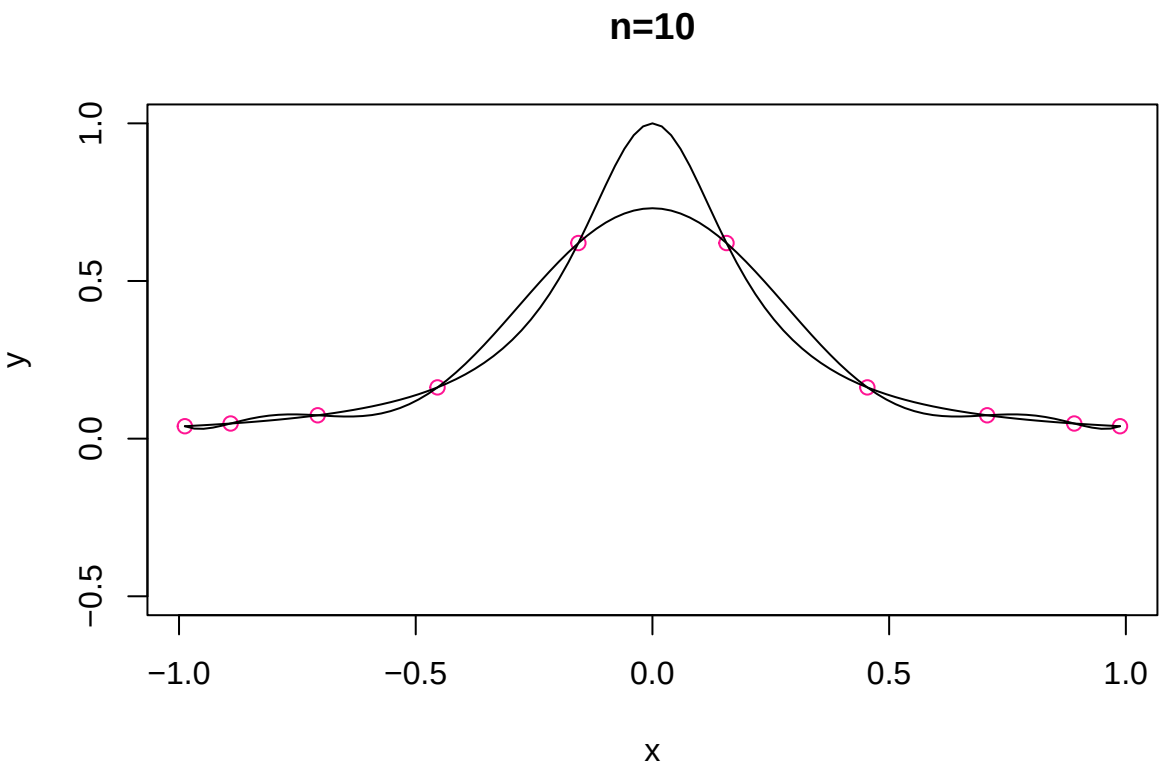
Interpolacja na brzegach stają się coraz dokładniejsze dla coraz większych wartości n.

c)

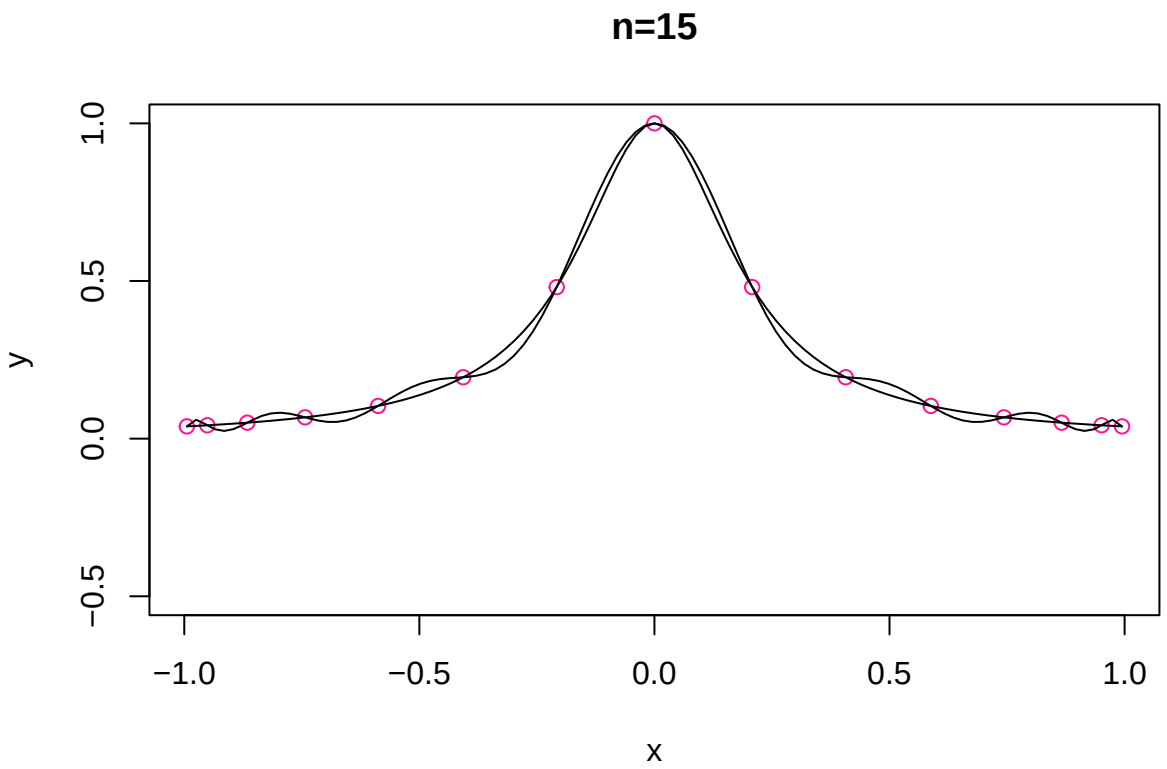
```
f<- function(x) 1/(1+25*(x^2))
n<-5
czeb<- function(x) cos((1+(0.5-x)/n)*pi)
x<-sapply(1:n,FUN=czeb)
y<-sapply(x,FUN=f)
f.p<-as.function(poly.calc(x,y))
plot(x,y,col="deeppink", main="n=5", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.p, add=TRUE)
```




```
n<-10
x<-sapply(1:n,FUN=czeb)
y<-sapply(x,FUN=f)
f.p<-as.function(poly.calc(x,y))
plot(x,y,col="deeppink", main="n=10", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.p, add=TRUE)
```




```
n<-15
x<-sapply(1:n,FUN=czeb)
y<-sapply(x,FUN=f)
f.p<-as.function(poly.calc(x,y))
plot(x,y,col="deeppink", main="n=15", ylim=c(-0.5,1))
curve(f, add=TRUE)
curve(f.p, add=TRUE)
```



Nie, nie widać znacznego pogorszenia interpolacji między węzłami znajdującymi się na brzegach przedziału.